МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ
УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
Кафедра «Интеллектуальные информационные технологии»

ЛАБОРАТОРНАЯ РАБОТА №3

по дисциплине «Основы машинного обучения»

Тема: «Сравнение алгоритмов классификации: k-NN, деревья решений, SVM»

Выполнил:
студент 3 курса
группы АС-67
Швед М. С.

Проверил:
Крощенко А. А.

Брест 2026

Цель: на практике сравнить работу нескольких алгоритмов классификации, таких как метод k-ближайших соседей (k-NN), деревья решений и метод опорных векторов (SVM). Научиться подбирать гиперпараметры моделей и оценивать их влияние на результат.

Общее задание:

1. Загрузить датасет по варианту;
2. Разделить данные на обучающую и тестовую выборки;
3. Обучить на обучающей выборке три модели: k-NN, Decision Tree и SVM;
4. Для модели k-NN исследовать, как меняется качество при разном количестве соседей (k);
5. Оценить точность каждой модели на тестовой выборке;
6. Сравнить результаты, сделать выводы о применимости каждого метода для данного набора данных.

Используемые инструменты: Python, Pandas, Matplotlib, scikit-learn, Orange Data Mining.

Задания по вариантам:

Вариант 1

- Iris
- Определить вид ириса (setosa, versicolor, virginica) по измерениям его цветка
- Задания:
  1. Загрузите данные и ознакомьтесь с ними;
  2. Разделите выборку на обучающую и тестовую;
  3. Обучите модели k-NN, Decision Tree и SVM. Для k-NN попробуйте найти оптимальное значение k;
  4. Оцените точность (accuracy) каждой модели на тестовой выборке;
  5. Сравните результаты и сделайте вывод, какая модель лучше всего справилась с этой задачей.

Код:

```
import os


import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
```

```
from sklearn.metrics import accuracy_score, confusion_matrix, ConfusionMatrixDisplay


BASE_DIR = os.path.dirname(os.path.abspath(__file__))


# 1. Загрузка и знакомство с данными
df = pd.read_csv(os.path.join(BASE_DIR, 'iris.csv'))


print("Первые 5 строк:")
print(df.head())
print(f"\nРазмер датасета: {df.shape}")
print(f"Пропусков: {df.isnull().sum().sum()}")
print("\nРаспределение классов:")
print(df['variety'].value_counts())
print("\nОписательная статистика:")
print(df.describe().round(2))


plt.figure(figsize=(8, 6))
for name, group in df.groupby('variety'):
    plt.scatter(group['petal.length'], group['petal.width'], label=name, alpha=0.7)
plt.title('Ирисы: длина и ширина лепестка')
plt.xlabel('petal.length')
plt.ylabel('petal.width')
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, 'iris_scatter.png'), dpi=150)
plt.show()


X = df.drop('variety', axis=1)
y = df['variety']


# 2. Разделение на обучающую и тестовую выборки
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)
print(f"\nОбучающая выборка: {len(X_train)}, тестовая: {len(X_test)}")


scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)


# 3. Подбор k для k-NN (кросс-валидация на обучающей выборке)
k_values = range(1, 31)
cv_scores = []
test_scores = []
for k in k_values:
    knn_k = KNeighborsClassifier(n_neighbors=k)
    cv_scores.append(cross_val_score(knn_k, X_train_scaled, y_train, cv=5).mean())
    knn_k.fit(X_train_scaled, y_train)
    test_scores.append(accuracy_score(y_test, knn_k.predict(X_test_scaled)))


best_k = list(k_values)[cv_scores.index(max(cv_scores))]
print("\nk-NN: точность в зависимости от k")
print(" k  | CV accuracy | Test accuracy")
for k, cv, test in zip(k_values, cv_scores, test_scores):
    print(f"{k:3d} |   {cv:.4f}    |   {test:.4f}")
print(f"Оптимальное k (по кросс-валидации): {best_k}")


plt.figure(figsize=(10, 6))
plt.plot(k_values, cv_scores, marker='o', label='Кросс-валидация (обучающая выборка)')
plt.plot(k_values, test_scores, marker='s', label='Тестовая выборка')
plt.axvline(best_k, color='red', linestyle='--', label=f'Оптимальное k = {best_k}')
plt.title('Точность k-NN в зависимости от количества соседей k')
plt.xlabel('k (количество соседей)')
plt.ylabel('Accuracy')
plt.legend()
```

```
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, 'knn_k.png'), dpi=150)
plt.show()


# 4. Обучение трёх моделей
knn = KNeighborsClassifier(n_neighbors=best_k)
dt = DecisionTreeClassifier(random_state=42)
svm = SVC(random_state=42)


knn.fit(X_train_scaled, y_train)
dt.fit(X_train_scaled, y_train)
svm.fit(X_train_scaled, y_train)


y_pred_knn = knn.predict(X_test_scaled)
y_pred_dt = dt.predict(X_test_scaled)
y_pred_svm = svm.predict(X_test_scaled)


# 5. Оценка точности на тестовой выборке
acc_knn = accuracy_score(y_test, y_pred_knn)
acc_dt = accuracy_score(y_test, y_pred_dt)
acc_svm = accuracy_score(y_test, y_pred_svm)


print(f"\nAccuracy k-NN (k={best_k}): {acc_knn:.4f}")
print(f"Accuracy Decision Tree: {acc_dt:.4f}")
print(f"Accuracy SVM: {acc_svm:.4f}")


# Дополнительно: кросс-валидация на всей выборке (устойчивее для 150 объектов)
print("\nСредняя точность по 5-кратной кросс-валидации (вся выборка):")
X_all_scaled = StandardScaler().fit_transform(X)
for name, model in [(f'k-NN (k={best_k})', KNeighborsClassifier(n_neighbors=best_k)),
                    ('Decision Tree', DecisionTreeClassifier(random_state=42)),
                    ('SVM', SVC(random_state=42))]:
    scores = cross_val_score(model, X_all_scaled, y, cv=5)
    print(f"{name}: {scores.mean():.4f} ± {scores.std():.4f}")


# 6. Сравнение
results = {f'k-NN (k={best_k})': acc_knn, 'Decision Tree': acc_dt, 'SVM': acc_svm}


plt.figure(figsize=(8, 5))
bars = plt.bar(results.keys(), results.values(), color=['steelblue', 'seagreen',
'darkorange'])
for bar, acc in zip(bars, results.values()):
    plt.text(bar.get_x() + bar.get_width() / 2, acc + 0.005, f'{acc:.3f}',
ha='center')
plt.ylim(0.8, 1.02)
plt.title('Сравнение точности моделей на тестовой выборке')
plt.ylabel('Accuracy')
plt.grid(axis='y', linestyle='--', alpha=0.6)
plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, 'models_comparison.png'), dpi=150)
plt.show()


fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
for ax, (name, y_pred) in zip(axes, [(f'k-NN (k={best_k})', y_pred_knn),
                                     ('Decision Tree', y_pred_dt), ('SVM',
y_pred_svm)]):
    cm = confusion_matrix(y_test, y_pred, labels=knn.classes_)
    ConfusionMatrixDisplay(cm, display_labels=knn.classes_).plot(ax=ax,
cmap=plt.cm.Blues, colorbar=False)
    ax.set_title(name)
plt.tight_layout()
plt.savefig(os.path.join(BASE_DIR, 'confusion_matrices.png'), dpi=150)
plt.show()
```

```
best_model = max(results.items(), key=lambda x: x[1])[0]
print(f"\nСамый эффективный метод на данном датасете - {best_model}")
```

Результаты выполнения программы:

1. Знакомство с данными

Датасет Iris содержит 150 объектов, 4 числовых признака (длина и ширина чашелистика и лепестка) и целевой признак variety. Пропусков нет, классы сбалансированы (по 50 объектов). Выборка разделена на обучающую (120) и тестовую (30) в соотношении 80/20 со стратификацией; признаки стандартизированы.

```
Первые 5 строк:
   sepal.length  sepal.width  petal.length  petal.width variety
0           5.1          3.5           1.4          0.2  Setosa
1           4.9          3.0           1.4          0.2  Setosa
2           4.7          3.2           1.3          0.2  Setosa
3           4.6          3.1           1.5          0.2  Setosa
4           5.0          3.6           1.4          0.2  Setosa


Размер датасета: (150, 5)
Пропусков: 0


Распределение классов:
variety
Setosa        50
Versicolor    50
Virginica     50
Name: count, dtype: int64


Описательная статистика:
       sepal.length  sepal.width  petal.length  petal.width
count        150.00       150.00        150.00       150.00
mean           5.84         3.06          3.76         1.20
std            0.83         0.44          1.77         0.76
min            4.30         2.00          1.00         0.10
25%            5.10         2.80          1.60         0.30
50%            5.80         3.00          4.35         1.30
75%            6.40         3.30          5.10         1.80
max            7.90         4.40          6.90         2.50


Обучающая выборка: 120, тестовая: 30
```

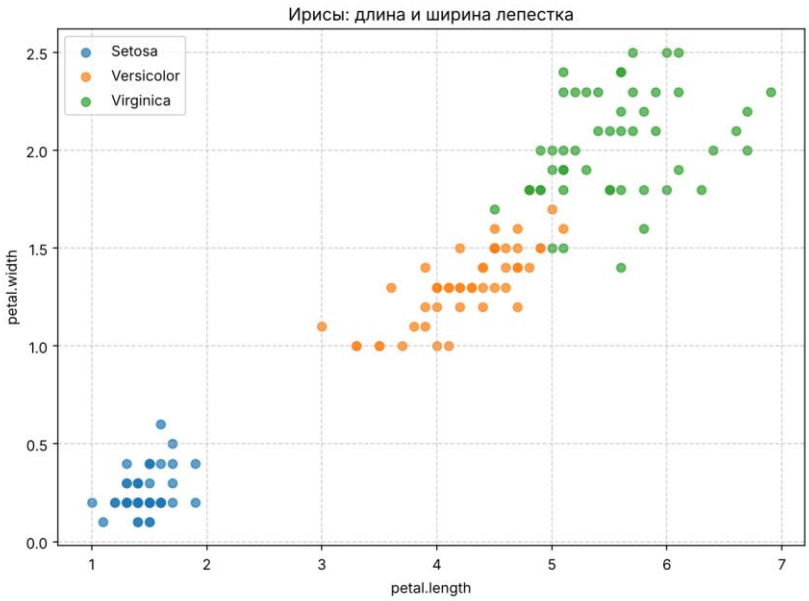


Рисунок 1 - Распределение классов по длине и ширине лепестка

Класс Setosa полностью отделяется от остальных по размерам лепестка, а Versicolor и Virginica частично пересекаются - именно на этих двух классах модели и ошибаются.

2. Подбор k для k-NN

Для k от 1 до 30 вычислена точность по 5-кратной кросс-валидации на обучающей выборке и точность на тестовой выборке. Оптимальное k выбиралось по кросс-валидации, чтобы не подгонять модель под тестовые данные.

```
k-NN: точность в зависимости от k
 k   | CV accuracy | Test accuracy
  1 |   0.9417    |   0.9667
  2 |   0.9583    |   0.9333
  3 |   0.9583    |   0.9333
  4 |   0.9583    |   0.9333
  5 |   0.9667    |   0.9333
  6 |   0.9667    |   0.9333
  7 |   0.9583    |   0.9667
  8 |   0.9583    |   0.9333
  9 |   0.9583    |   0.9667
 10 |   0.9667    |   0.9667
 11 |   0.9583    |   0.9667
 12 |   0.9667    |   0.9667
 13 |   0.9500    |   0.9667
 14 |   0.9583    |   0.9667
 15 |   0.9500    |   0.9667
 16 |   0.9583    |   0.9667
 17 |   0.9583    |   0.9667
 18 |   0.9417    |   0.9667
 19 |   0.9417    |   0.9667
 20 |   0.9333    |   0.9667
 21 |   0.9333    |   0.9667
 22 |   0.9250    |   0.9333
 23 |   0.9250    |   0.9333
 24 |   0.9083    |   0.9000
 25 |   0.8917    |   0.9000
 26 |   0.9083    |   0.8667
 27 |   0.9000    |   0.8667
```

```
 28 |    0.8917     |    0.8667
 29 |    0.8917     |    0.8667
 30 |    0.8917     |    0.8667
Оптимальное k (по кросс-валидации): 5
```

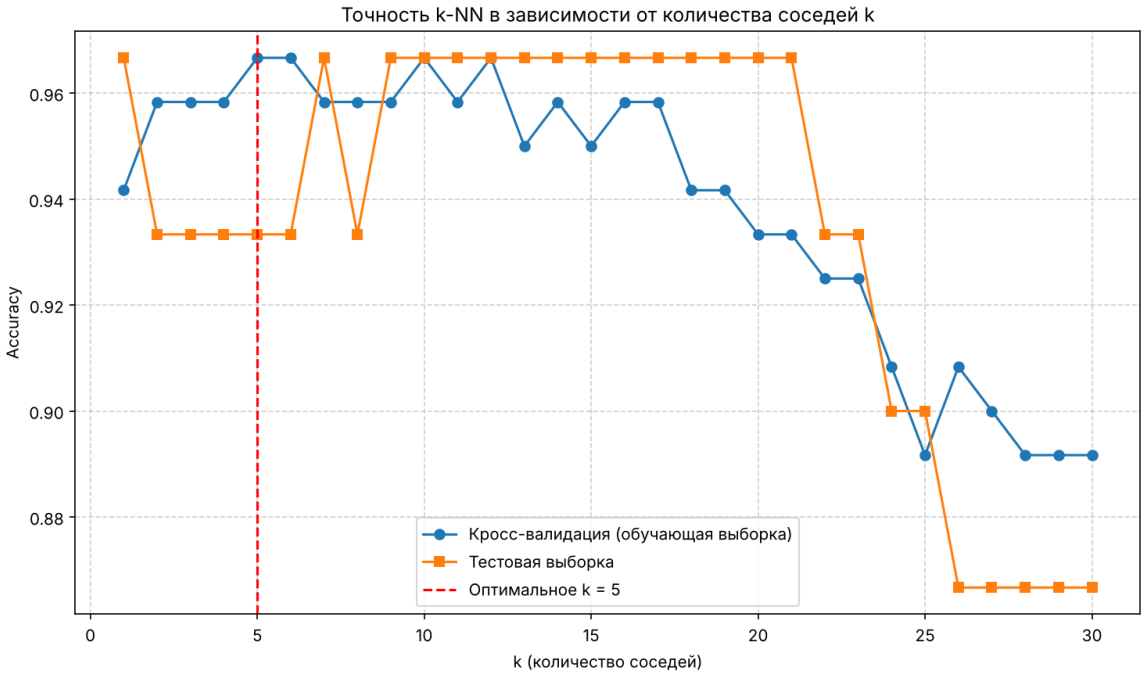


Рисунок 2 - Зависимость точности k-NN от количества соседей k

При малых k (k = 1) модель чувствительна к шуму и отдельным объектам (переобучение), CV accuracy = 0,942. При k от 5 до 12 точность максимальна и стабильна (≈ 0,96-0,97). При k > 20 точность быстро падает до 0,87-0,89: соседей становится так много, что в голосовании участвуют объекты чужих классов (недообучение). Оптимальное значение - k = 5.

3. Сравнение моделей

```
Accuracy k-NN (k=5): 0.9333
Accuracy Decision Tree: 0.9333
Accuracy SVM: 0.9667


Средняя точность по 5-кратной кросс-валидации (вся выборка):
k-NN (k=5): 0.9600 ± 0.0249
Decision Tree: 0.9533 ± 0.0340
SVM: 0.9667 ± 0.0211


Самый эффективный метод на данном датасете - SVM
```

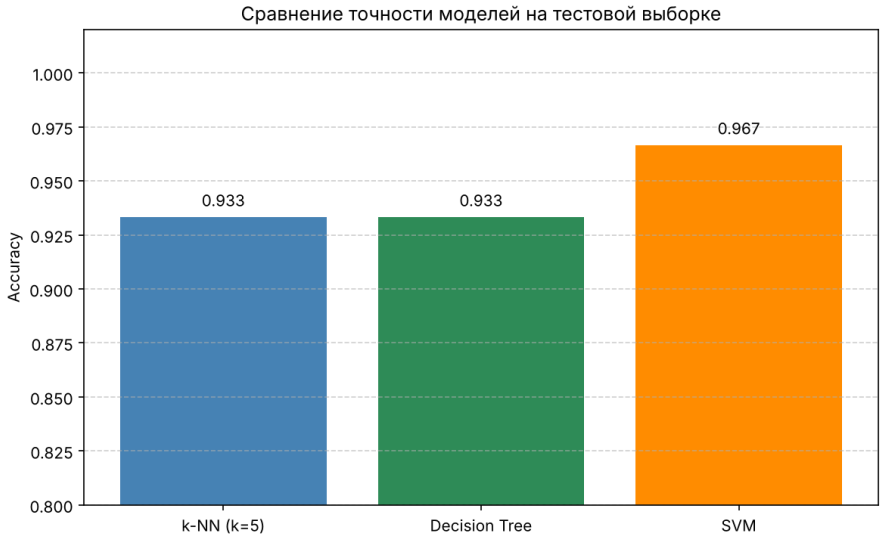


Рисунок 3 - Точность моделей на тестовой выборке

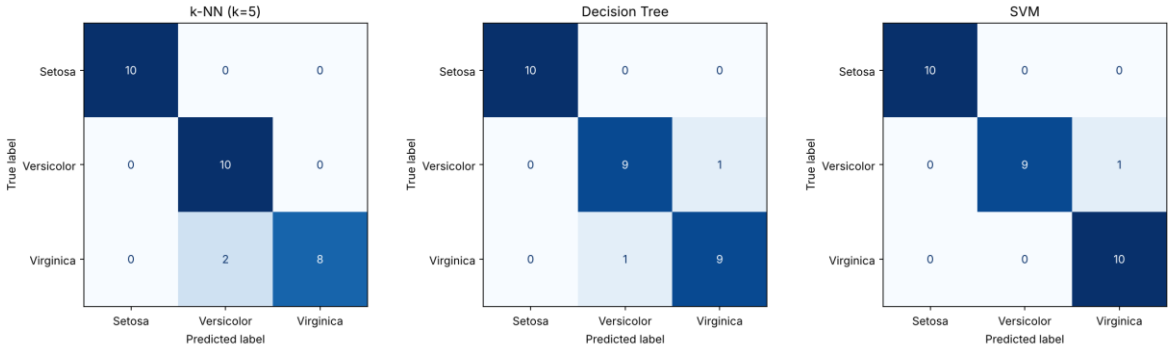


Рисунок 4 - Матрицы ошибок моделей

Все три модели безошибочно определяют Setosa. Ошибки возникают только между Versicolor и Virginica: k-NN ошибся 2 раза (две Virginica отнесены к Versicolor), дерево решений - 2 раза, SVM - 1 раз. Лучший результат на тестовой выборке показал SVM (accuracy = 0,967), k-NN и Decision Tree - по 0,933. Так как тестовая выборка мала (30 объектов, одна ошибка = 3,3 %), дополнительно проведена 5-кратная кросс-валидация на всей выборке: SVM - 0,967 +- 0,021, k-NN - 0,960 +- 0,025, Decision Tree - 0,953 +- 0,034. Порядок моделей сохраняется.

Вывод: в ходе лабораторной работы я сравнил алгоритмы k-NN, Decision Tree и SVM на датасете Iris и исследовал влияние гиперпараметра k на качество k-NN. Слишком маленькое k приводит к переобучению, слишком большое - к недообучению; оптимальным оказалось k = 5. Все модели показали высокую точность (93–97 %), так как классы хорошо разделяются по размерам лепестка. Лучше всего справился SVM (accuracy = 0,967 на тесте и 0,967 по кросс-валидации): он строит гладкую разделяющую границу между близкими классами Versicolor и Virginica. k-NN хорошо подходит для небольших данных с малым числом признаков, но требует нормализации и

подбора k. Дерево решений легко интерпретировать, но оно склонно к переобучению и даёт наибольший разброс результатов.